Standard support for Java 8 and 11 ends in August 2026 for 4.6 LTS. Plan your upgrade path for apps that are running on Java 8 or 11 accordingly.

Transactions

Implementing Transactions in a Connector

The SDK supports the creation of connectors that join and create a new transaction in a Mule Flow, including Source and Operation executions that work as a unit. So, if a problem occurs inside the flow, all the components participating in the transaction will roll back their execution as a unit. All operations within the transaction are rolled back so that no one part results in partial completion.

Implementing Transactions in a Connector

Mule transaction support works on the premise of a transactional resource identified by a key. The SDK follows this same concept of reusing the connection abstraction and using the component connection as the key of the transaction.

An extension must have a `ConnectionProvider` in order to have a transaction beca the `ConnectionProvider` yields instances of the `TransactionalConnection` interface.

Troubleshooting

TransactionalConnection Interface

To enable transactions in your connector, you need a Connection Provider (`ConnectionProvider`) because it provides connections that implement the `TransactionalConnection` interface.

The connection must implement `TransactionalConnection` methods that are used to communicate the beginning and conclusion of a transaction.

JAVA |

```java
public class TXConnectionProvider implements ConnectionProvider<TXConnection> {
  ...
}
```

JAVA |

```java
public class TxConnection implements TransactionalConnection {

    @Override
    public void begin() throws TransactionException {
        //logic to begin a transaction
    }

    @Override
    public void commit() throws TransactionException {
        //logic to commit a transaction

    }

    @Override
    public void rollback() throws TransactionException {
        //logic to rollback a transaction
```

```
    }
}
```

Implementing this interface requires implementation of the following methods:

Method	Description
`begin()`	This method is called, when the transaction must begin.
`commit()`	This is called when the transaction finished correctly and must be committed.
`rollback()`	This is called then the transaction finished with an error and this one must be rollbacked.

Transactions in Operations

To make an operation work under a transaction, the `@Connection` parameter should be a transactional-capable connection, that is, a connection that implements `TransactionalConnection`.

Transactional Actions

When an operation is transactional, a synthetic parameter named `transactionalAction` is automatically added to it. That parameter is an Enum type that can take these values:

Action Name	Description
ALWAYS_JOIN	The connector will always join the existing transaction. If there is no transaction, or if the current transaction is not compatible with the

Action Name	Description
	current operation, the operation will fail to execute.
JOIN_IF_POSSIBLE (Default)	If a transaction is available, use it. Otherwise, continue processing.
NOT_SUPPORTED	The operation will execute outside any ongoing transaction.

XML

```xml
<try transactionalAction="BEGIN_OR_JOIN">
  <transactional-connector:work config-ref="config"
    transactionalAction="ALWAYS_JOIN"/>
</try>
```

Operation Transactional Actions Inside the Operation Code

In some cases, you might want to know the operation level at which `transactionalAction` has been configured in the Mule app. You can do so by setting a parameter of `OperationTransactionalAction` type to inject the used value.

```java
public void work(@Connection TxConnection connection, OperationTransactionalAction ac
  //
}
```

Operation Transaction Usage

Operations cannot start transactions. They can be started by a Message Source, or the operation has to be inside a `<try />` scope that starts a new transaction.

Prior to executing a transactional operation, Mule will check if the current transaction has a bound key:

- If there is already a bounded key (because a previous Operation or Source has already registered), Mule will check for its compatibility with the current Operation. If it is compatible, the Operation will use the bounded connection. Otherwise, the execution flow will fail, indicating that the current Operation was trying to enter to an incompatible Transaction.
- If not, it will bound the Operation's Connection as the key and execute the `TransactionalConnection.begin()`, communicating that the transaction for the current Operation should start.

At the end of the execution of the transaction scope (meaning the `<try />` scope or the entire flow if the transaction was initialized by a Message Source), Mule will call

`TransactionalConnection.commit()` if the transaction finished correctly. Otherwise, if an error occurred, it will execute `TransactionalConnection.rollback()`.

When the connection joins the transaction, the provider's connection management strategy is overridden, meaning that regardless of the connection's strategy, the same connection will be used for all matching operations between the transaction's scope.

> **WARNING**
> This behavior is not totally compatible with the concept of `CachedConnectionProvider` because that would allow the same connection to join different transactions concurrently. This implies that you should know how to manage different transactions simultaneously in the same connection. The use of `CachedConnectionProviders` with Transactions is strongly discouraged.

Transactions in Sources

To make an Operation work under a transaction, the `@Connection` parameter should be a `ConnectionProvider` capable of providing transaction-compatible connections.

Transactional Actions

Source Transactional Actions

When a Message Source is transactional, a synthetic parameter named `transactionalAction` is automatically added to it. That parameter is an Enum type

that can take these values:

Table 1. Source Transactional Actions

Action Name	Description
ALWAYS BEGIN	Ensures that a new transaction is created for each invocation.
NONE (Default)	The source will not start any transaction and will not participate in one that is open in the Flow.

Source Transactional Actions inside the Operation Code

In some cases, you might want to know at Source level which `transactionalAction` has been configured in the Mule app. If so, you can define a parameter of `SourceTransactionalAction` type to inject the used value, for example:

Injecting SourceTransactionalAction to Source

```java
public class TransactionalSource extends Source<String, Void> {

  @Connection
  private ConnectionProvider<TXConnection> connection;

  @Parameter
  private SourceTransactionalAction action;

  // rest of the code
}
```

More more on this topic, Sources Transactions.

XA Transactions

XA transactions are supported in a similar way. The only difference is that instead of a `TransactionalConnection`, the provider should return an `XATransactionalConnection`, which is a connection capable of returning an `XAResource`.

Back to top

DID THIS ARTICLE SOLVE YOUR ISSUE?
Let us know so we can improve!

Yes No

Legal | Terms of Service | Privacy | Trust | Contact | Responsible Disclosure | Cookie Settings
Your Privacy Choices

© Copyright 2026 Salesforce, Inc. All rights reserved. Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States